

Docker Fundamentals

ML Engineer Journey · Phase 3 — Docker & Clean Architecture · Week 09

SESI 1 — Container vs VM: Konsep Mendalam

1. Analogi: Gedung vs Partisi

Bayangkan kamu mau buka 3 cabang kafe. Dengan **Virtual Machine**, kamu bangun 3 gedung terpisah dari nol — setiap gedung punya fondasi, listrik, plumbing, dan satpam sendiri. Terisolasi sempurna, tapi mahal dan butuh waktu lama untuk dibangun.

Dengan **Container**, kamu sewa satu gedung besar dan pasang partisi-partisi ringan. Setiap partisi punya menu dan staf sendiri, tapi berbagi fondasi, listrik, dan dapur besar yang sama. Startup-nya cepat — pasang sekat, langsung buka.

2. Perbedaan Fundamental

Perbedaan paling mendasar: **VM membawa Guest OS sendiri**, container tidak. Container berbagi kernel Host OS langsung. Dari sinilah semua perbedaan lain muncul — ukuran, kecepatan startup, overhead resource.

Aspek	Virtual Machine	Container
Isolasi	Hardware-level (Hypervisor)	OS-level (Namespaces + cgroups)
OS	Tiap VM punya Guest OS sendiri	Berbagi kernel Host OS
Ukuran	GB (bawa OS penuh)	MB (hanya app + libs)
Startup	Menit	Detik (bahkan milidetik)
Overhead	Tinggi	Rendah
Keamanan isolasi	Sangat kuat (hardware boundary)	Kuat, tapi lebih tipis
Portabilitas	Terbatas (image besar)	Tinggi (image kecil, standar OCI)
Cocok untuk	Multi-tenant cloud, OS berbeda	Microservices, CI/CD, ML serving

3. Dua Fitur Linux yang Membuat Container Berjalan

Namespaces — memberikan ilusi isolasi. Setiap container punya 'view' sendiri terhadap sistem: PID namespace (proses tidak bisa lihat proses container lain), network namespace (interface jaringan sendiri),

mount namespace (filesystem sendiri). Container merasa seolah-olah berjalan di machine sendiri.

cgroups (control groups) — membatasi resource. Menentukan berapa CPU, RAM, dan disk I/O yang boleh dipakai container. Tanpa cgroups, satu container bisa 'memakan' semua RAM host dan merusak container lain.

Kesimpulan penting: Container bukan 'VM yang lebih ringan.' Container adalah proses Linux yang diisolasi dengan sangat ketat. Fundamentalnya berbeda.

4. Layer System — Cara Image Docker Disusun

Container berjalan dari sebuah **image** — snapshot beku dari filesystem yang tersusun dalam layer-layer bertumpuk:

Layer	Isi	Sifat
Container layer	Writable — dibuat saat run	Ephemeral (hilang saat container dihapus)
Layer 4	App code (COPY . .)	Read-only · ~1 MB
Layer 3	pip dependencies	Read-only · ~200 MB · ter-cache
Layer 2	Python 3.11 runtime	Read-only · ~50 MB · ter-cache
Layer 1	Base OS (debian:slim)	Read-only · ~30 MB · ter-cache

Layer read-only di-share antar container yang pakai image sama. 10 container dari image yang sama tidak butuh 10x storage. Hanya container layer yang writable — dan hilang saat container dihapus. Data persisten harus disimpan di **volume** (W10).

5. Relevansi Langsung ke Project Kamu

Selama Phase 2, Railway mengelola environment secara transparan — kamu tidak tahu container apa yang dipakai, konfigurasinya seperti apa, atau versi Python yang digunakan. Docker memberimu kendali penuh atas hal itu.

Tanpa Docker: dev machine kamu memakai Python 3.11 + scikit-learn 1.3.0, Railway memakai Python 3.10 + scikit-learn 1.2.2 — 'works on my machine' yang klasik. Dengan Docker: image yang sama jalan di mana pun. Environment identik. Reproducibility adalah janji utama container.

SESI 2 — Dockerfile: Instruksi Dasar

1. Analogi: Dockerfile sebagai Resep

Setiap kali setup project baru, kamu lakukan langkah berurutan: buat folder, buat venv, install dependencies, copy kode, jalankan app. **Dockerfile adalah resep tertulis dari langkah-langkah itu.** Docker membacanya dan mengeksekusi secara otomatis, deterministik, di machine manapun. Urutan penting, bahan harus tepat, hasilnya harus sama setiap kali.

2. Instruksi-Instruksi Dockerfile

Instruksi	Kapan dijalankan	Fungsi	Contoh
FROM	Build-time (pertama)	Tentukan base image	FROM python:3.11-slim
WORKDIR	Build-time	Set working directory	WORKDIR /app
COPY	Build-time	Salin file host → image	COPY requirements.txt .
RUN	Build-time	Eksekusi command, buat layer	RUN pip install -r requirements.txt
EXPOSE	Build-time (metadata)	Dokumentasi port (deklaratif)	EXPOSE 8000
CMD	Runtime (saat docker run)	Default command saat container start	["uvicorn", "main:app"]

3. RUN vs CMD — Perbedaan Kritis

	RUN	CMD
Kapan dijalankan	Saat build image	Saat container start (docker run)
Berapa kali	Sekali — hasilnya jadi layer	Setiap container start
Tujuan	Install deps, setup environment	Jalankan aplikasi
Contoh	pip install, apt-get install	uvicorn main:app

Analogi: RUN = persiapan dapur sebelum restoran buka. CMD = masak ketika ada pelanggan datang.

4. Dockerfile Lengkap — FastAPI App

```
# Layer 1: Base image – Python 3.11 versi slim (tanpa bloat)
FROM python:3.11-slim

# Layer 2: Set working directory di dalam container
WORKDIR /app

# Layer 3: Copy requirements DULU – sebelum kode app (untuk layer caching)
COPY requirements.txt .

# Layer 4: Install dependencies
```

```
# --no-cache-dir: jangan simpan cache pip di image (hemat space)
RUN pip install --no-cache-dir -r requirements.txt

# Layer 5: Copy seluruh kode app ke dalam image
COPY . .

# Dokumentasi: container ini listen di port 8000 (hanya metadata)
EXPOSE 8000

# Command yang dijalankan saat container start
# Gunakan format JSON array (exec form) – bukan string
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

5. Layer Caching — Kenapa Urutan COPY Penting

Ini adalah salah satu optimasi paling penting dalam menulis Dockerfile. Docker meng-cache setiap layer. Jika sebuah layer berubah, **semua layer di bawahnya ikut di-invalidate** dan harus di-rebuild.

Cara naif — salah:

```
# COPY . .          ← jika kode berubah, layer ini invalid
# RUN pip install    ← pip install dijalankan ulang dari nol tiap kali!
```

Cara yang benar — pisahkan requirements dari kode:

```
# COPY requirements.txt . ← hanya invalid jika requirements berubah
# RUN pip install         ← ter-cache selama requirements sama
# COPY . .               ← kode sering berubah, tapi pip tidak diulang
```

Hasil nyata dari latihan hari ini: pip install yang butuh 17.8 detik di build pertama menjadi 0.0 detik (CACHED) di build kedua setelah mengubah kode.

6. Kenapa --host 0.0.0.0 di CMD

Di Phase 2, uvicorn dijalankan di 127.0.0.1 (localhost). Di dalam container, 127.0.0.1 artinya 'hanya dari dalam container itu sendiri' — koneksi dari laptop tidak bisa masuk.

0.0.0.0 artinya 'terima koneksi dari semua interface' sehingga koneksi dari laptop ke container bisa masuk melalui port mapping. Ini bukan security issue — kamu tetap kontrol via **-p flag** saat docker run. 0.0.0.0 di dalam container tidak berarti terbuka ke internet.

7. Command Docker yang Wajib Diketahui

Command	Fungsi
<code>docker build -t nama .</code>	Build image dari Dockerfile di direktori saat ini
<code>docker run -d -p 8000:8000 nama</code>	Jalankan container di background, map port
<code>docker images</code>	Lihat semua image yang ada di local
<code>docker ps</code>	Lihat container yang sedang berjalan
<code>docker ps -a</code>	Lihat semua container (termasuk yang stopped)

<code>docker logs nama</code>	Lihat log output dari container
<code>docker stop nama</code>	Stop container yang berjalan
<code>docker rm nama</code>	Hapus container (bukan image)
<code>docker rmi nama</code>	Hapus image

Key Takeaways

1. Container BUKAN VM ringan — fundamentalnya berbeda. VM punya Guest OS sendiri; container berbagi kernel Host OS dan diisolasi via Namespaces + cgroups.
2. Namespaces = isolasi logis (tiap container punya view sendiri). cgroups = limit resource (CPU, RAM, I/O).
3. Layer system: image terdiri dari layer read-only yang ter-cache. Container layer adalah satu-satunya yang writable — dan ephemeral.
4. Container tidak bisa lintas kernel: container Linux butuh Host Linux. Container Windows butuh Host Windows.
5. Urutan instruksi Dockerfile menentukan efisiensi caching. Selalu COPY requirements.txt dulu, baru COPY kode.
6. RUN dijalankan saat BUILD. CMD dijalankan saat RUNTIME (docker run).
7. EXPOSE hanya metadata — tidak otomatis publish port. Gunakan -p saat docker run.
8. 0.0.0.0 di dalam container = terima koneksi dari semua interface container, bukan berarti terbuka ke internet.

github.com/lolivampire/ML-Project · Next: W09D03 — docker build & docker run (hands-on Project 1)